OpenBao Security Evolution Analysis Report

Versions 2.4.0 to 2.7.2

Adrien SALES, geol, Trivy and Claude Code

November 1, 2026

Abstract

This document provides a comprehensive analysis of the security evolution in OpenBao, covering major and minor releases from version 2.4.0 to 2.7.2.

Utilizing the Trivy vulnerability scanner on the respective OCI images, this report highlights the continuous improvement in the security posture of the project. To ensure a methodologically sound comparison, **all nineteen image versions were rescanned in a single session against an identical Trivy vulnerability database** (2026-10-31), eliminating the database-drift bias that arises when versions are scanned weeks apart. The analysis encompasses vulnerability trends by severity and provides insights into the impact of base image updates (Alpine Linux) on the overall risk profile. This edition also identifies and corrects a scanner artifact — Go pseudo-version matching — that inflated every 2.x build up to v2.6.5 with seven already-fixed OpenBao core advisories, and which an earlier edition of this report misread as a code fix in v2.7.0. The findings demonstrate a significant and consistent reduction in critical and high-severity vulnerabilities, validating the project's commitment to security and the importance of adopting the latest stable releases.

1 Executive Summary

1.1 Key Security Metrics at a Glance

Critical	High	Medium	Low
0	**2**	**4**	**0**
v2.7.2	v2.7.2	v2.7.2	v2.7.2
(was 9 in v2.4.0)	(was 88 in v2.4.0)	(was 98 in v2.4.0)	(was 67 in v2.4.0)

1.2 Key Takeaways

- **Lowest Vulnerability Count:** v2.7.2 — **9** total findings (0 Critical, 2 High, 4 Medium, 0 Low, 3 Unknown), Security Score: **98.2/100**. The six weighted findings are three `go-jose` advisories (each reported in both the v3 and v4 modules) published on 2026-10-26 — the very day v2.7.2 was built.
- **Highest Vulnerability Count:** v2.4.0/v2.4.1 — 268 total vulnerabilities, 9 Critical, Security Score: **20.7/100**
- **Biggest Improvement:** v2.7.1 → v2.7.2 — **71.0% reduction** (31 → 9): Go toolchain 1.27.1 → 1.27.3, `golang.org/x/net` 0.58 → 0.61 and the Alpine 3.24.3 rebase clear every stdlib, HTTP/2 and OS finding. The parallel 2.6 patch, v2.6.4 → v2.6.5, cuts 59.0% (39 → 16).
- **Correction to the previous edition:** the seven OpenBao core advisories that earlier editions reported as "retired by v2.7.0" were **never present** in v2.5.4 and later — they are a Go pseudo-version matching artifact (Section 7.2). Once removed, **v2.6.5 and v2.7.2 carry the identical nine findings** and the same 98.2/100 score; the 2.6 and 2.7 lines are equally secure.
- **Recommended Minimum:** v2.6.5 or v2.7.2 for production use (Score $\geq$ 95.3/100 raw, 98.2/100 corrected). Both shipped 2026-10-26.
- **CVE database drift, measured again:** v2.7.0 was published in the previous edition with **1** finding; rescanned today, the unchanged image reports **31**. Comparing editions naively makes v2.7.2 (9) look like a regression from v2.7.0 (1), when it is in fact a 22-finding improvement (Section 8).
- **Overall Improvement: 96.6% reduction** from v2.4.0 to v2.7.2 (268 → 9 vulnerabilities)
- **Vulnerability Remediation:** 94–98% of findings are fixable across v2.4.0–v2.6.2, falling to 56% in v2.7.2. Of v2.7.2's nine findings, five have an upstream fix (`go-jose/v4` 4.1.6, `x/net` 0.62.0) and four do not (`go-jose/v3`, which has no patched release, and GO-2026-5932).
- **Versioning Policy:** OpenBao follows Semantic Versioning. Upgrading from v2.5.x to v2.5.y is **strongly encouraged** and considered low-risk. These updates are essential to benefit from security patches, bug fixes, and performance improvements without breaking compatibility. Staying on the latest patch of a minor release is the recommended security best practice.

1.3 Security Posture Scores

The Security Posture Score weights each finding by severity (Critical: 10 points, High: 5, Medium: 2, Low: 1) and expresses the result against a **fixed reference ceiling of 1000 weighted points**:

$$\text{Score} = 100 - \frac{\text{weighted points}}{1000} \times 100$$

The choice of a fixed ceiling is deliberate. Earlier editions of this report normalized against the *worst version in the corpus*, which had three defects: the worst version always scored exactly 0 by construction

rather than by measurement, every score silently re-scaled whenever a version was added or the CVE database moved, and scores could not be compared against any other product's report. With a constant denominator, a score means the same thing in every edition and for every product, and no version is pinned to either endpoint — v2.4.0 scores 20.7 rather than a definitionally-zero 0.0, which is the more truthful statement that it is bad but not maximally bad.

Table 1: Security Posture Score by Version (raw Trivy findings)

Version	Critical	High	Medium	Low	Total	Weighted	Score /100
2.4.0	9	88	98	67	262	793	**20.7**
2.4.1	9	88	98	67	262	793	**20.7**
2.4.3	9	87	81	64	241	751	24.9
2.4.4	9	87	79	64	239	747	25.3
2.5.0	9	80	68	45	202	671	32.9
2.5.1	8	79	68	45	200	656	34.4
2.5.2	6	78	67	43	194	627	37.3
2.5.3	1	54	56	38	149	430	57.0
2.5.4	1	48	53	38	140	394	60.6
2.5.5	1	32	35	16	84	256	74.4
2.6.0	0	31	32	14	77	233	76.7
2.6.1	0	29	32	14	75	223	77.7
2.6.2	0	19	31	14	64	171	82.9
2.6.3	0	12	20	2	34	102	89.8
2.6.4	0	12	20	2	34	102	89.8
2.6.5	0	7	6	0	13	47	**95.3**
2.7.0	0	7	18	1	26	72	92.8
2.7.1	0	7	18	1	26	72	92.8
2.7.2	0	2	4	0	6	18	**98.2**

Note on comparability with earlier editions: scores in this edition differ from previously published figures because the CVE database is five weeks newer (Section 8); no image changed. Scores within this report are mutually comparable; scores quoted from an earlier edition are not. Scores for v2.4.0–v2.6.5 also include the pseudo-version false positives described in Section 7.2; corrected scores are given there.

1.4 Risk Level Matrix

This matrix provides a quick visual assessment of the risk level associated with each version based on vulnerability composition.

Table 2: Risk Level Assessment by Version

Version	Critical	High	Medium	Low	Risk Level
2.4.0	9	88	98	67	**HIGH**
2.4.1	9	88	98	67	**HIGH**
2.4.3	9	87	81	64	**HIGH**
2.4.4	9	87	79	64	**HIGH**
2.5.0	9	80	68	45	**HIGH**
2.5.1	8	79	68	45	**HIGH**
2.5.2	6	78	67	43	**HIGH**
2.5.3	1	54	56	38	**MEDIUM**
2.5.4	1	48	53	38	**MEDIUM**
2.5.5	1	32	35	16	**MEDIUM**
2.6.0	0	31	32	14	**LOW**
2.6.1	0	29	32	14	**LOW**
2.6.2	0	19	31	14	**LOW**
2.6.3	0	12	20	2	**LOW**
2.6.4	0	12	20	2	**LOW**
2.6.5	0	7	6	0	**LOW**
2.7.0	0	7	18	1	**LOW**
2.7.1	0	7	18	1	**LOW**
2.7.2	0	2	4	0	**LOW**

Risk Level Guidelines:

- **HIGH** (Score < 45, i.e. more than 550 weighted points): Not recommended for production. Contains critical vulnerabilities requiring immediate attention.
- **MEDIUM** (Score 45–75, i.e. 250–550 weighted points): Use with caution. Suitable for development environments with proper monitoring.
- **LOW** (Score > 75, i.e. fewer than 250 weighted points): Recommended for production. Minimal security concerns with available patches.

2 Introduction

This report provides an in-depth analysis of the security evolution in OpenBao from version 2.4.0 through 2.7.2. The assessment was performed using the Trivy vulnerability scanner on the official OCI images for each release. The primary objective is to track the evolution of the project's security posture over time, identifying trends in vulnerability counts across different severity levels and evaluating the impact of maintenance and updates, such as base operating system upgrades.

2.1 OpenBao Versioning Policy and Release Strategy

OpenBao adheres to **Semantic Versioning** principles, ensuring predictable and transparent release management. The project follows a versioning scheme similar to industry standards, where version numbers follow the pattern `MAJOR.MINOR.PATCH`.

Similar to PostgreSQL's versioning policy[1], OpenBao's minor releases are designed to be **low-risk updates** that focus on:

- Fixes for frequently-encountered bugs
- Low-risk stability improvements
- Security vulnerability patches
- Data corruption prevention

The OpenBao community considers performing minor upgrades to be less risky than continuing to run an old minor version. This philosophy is validated by the security analysis presented in this report, where consistent minor version updates demonstrate significant vulnerability reduction without introducing breaking changes.

As PostgreSQL documentation states: *"Minor releases only contain fixes for frequently-encountered bugs, low-risk fixes, security issues, and data corruption problems. The community considers performing minor upgrades to be less risky than continuing to run an old minor version."* This same principle applies to OpenBao's release strategy, making the adoption of latest minor releases a recommended security best practice.

2.2 Lifecycle Status

According to **geol**, the 2.7 cycle (released 2026-09-23) is current, with v2.7.2 as its latest patch. The 2.6 cycle is listed as end-of-life from **2026-10-01** — yet the project shipped v2.6.4 that same day and v2.6.5 on 2026-10-26, each in lockstep with a 2.7 patch. Users still on 2.6.x have therefore continued to receive security rebuilds past the listed EOL date, but should not rely on this continuing; migrating to 2.7.x is the durable path.

3 Tooling Details

This comparative analysis report was generated and compiled using several tools, including an AI agent (Claude Code), to assess and present the vulnerability data.

3.1 Claude Code

This entire report, including the analysis, data extraction, and LaTeX document generation, was orchestrated by an instance of Claude Code. The agent was responsible for:

1. Interacting with vulnerability scanning tools.
2. Parsing their outputs.

[1] PostgreSQL Versioning Policy: https://www.postgresql.org/support/versioning/

3. Performing comparative data analysis.
4. Constructing the LaTeX document structure.
5. Generating the textual content and charts.
6. Compiling the LaTeX document into a PDF.
7. Obtaining End-of-Life (EOL) information for OpenBao versions from Geol.

3.2 Geol

Geol is a tool used for providing product lifecycle and support information.

- **Version:** 2.21.5
- **Usage:** Querying OpenBao release lifecycles and support status.

3.3 Trivy

Trivy (v0.76.1) is a comprehensive security scanner for container images. It was the primary tool for vulnerability assessment in this report. **All nineteen image tags were rescanned on 2026-11-01 against a single, shared vulnerability database** (downloaded once with `--download-db-only`, then reused via `--skip-db-update`), so that every version is evaluated against the exact same set of known CVEs.

- **Vulnerability Database Updated:** 2026-10-31 18:53 UTC
- **Scanning Commands Examples:**

```
trivy image --format json --output openbao_v2.4.0.json openbao/openbao:2.4.0
trivy image --format json --output openbao_v2.7.2.json openbao/openbao:2.7.2
```

- **Analyzed Image Tags:** 2.4.0, 2.4.1, 2.4.3, 2.4.4, 2.5.0, 2.5.1, 2.5.2, 2.5.3, 2.5.4, 2.5.5, 2.6.0, 2.6.1, 2.6.2, 2.6.3, 2.6.4, 2.6.5, 2.7.0, 2.7.1, 2.7.2.

4 Tool Homepages

- **Claude Code:** https://claude.com/claude-code
- **Geol:** https://github.com/opt-nc/geol
- **Trivy:** https://trivy.dev/

5 Base Image Analysis

The security posture of an OCI image is heavily influenced by its base operating system. OpenBao utilizes Alpine Linux, a security-oriented, lightweight Linux distribution based on musl libc and busybox.

5.1 Base OS Freshness

Using the newest Alpine patch available is as important as the choice of major/minor branch: Alpine ships synchronized security-patch releases across all supported branches roughly every 4–10 weeks. For each OpenBao release, we cross-referenced **geol** (Alpine Linux cycle/EOL data) with the exact **Docker Hub tag publish timestamps** for every `alpine:3.2x.y` tag to reconstruct which Alpine patch was actually the latest one publicly available on the day each OpenBao image was built — not merely the latest one known today (the two can differ, since new Alpine patches are released continuously).

Table 3: Base OS Versions (Alpine Linux) and Freshness at Release Time

OpenBao Version	Base OS	Version Used	Latest Alpine Available (that day)	Latest at Release?
2.4.0	Alpine	3.22.1	3.22.1	**Yes**
2.4.1	Alpine	3.22.1	3.22.1	**Yes**
2.4.3	Alpine	3.22.2	3.22.2	**Yes**
2.4.4	Alpine	3.22.2	3.22.2	**Yes**
2.5.0	Alpine	3.23.3	3.23.3	**Yes**
2.5.1	Alpine	3.23.3	3.23.3	**Yes**
2.5.2	Alpine	3.23.3	3.23.3	**Yes**
2.5.3	Alpine	3.23.4	3.23.4	**Yes**
2.5.4	Alpine	3.23.4	3.23.4	**Yes**
2.5.5	Alpine	3.24.1	3.24.1	**Yes**
2.6.0	Alpine	3.24.1	3.24.1	**Yes**
2.6.1	Alpine	3.24.1	3.24.1	**Yes**
2.6.2	Alpine	3.24.1	3.24.1	**Yes**
2.6.3	Alpine	3.24.2	3.24.2	**Yes**
2.6.4	Alpine	3.24.2	3.24.2	**Yes**
2.6.5	Alpine	3.24.3	3.24.3	**Yes**
2.7.0	Alpine	3.24.2	3.24.2	**Yes**
2.7.1	Alpine	3.24.2	3.24.2	**Yes**
2.7.2	Alpine	3.24.3	3.24.3	**Yes**

Finding: every single analyzed OpenBao release — all nineteen — shipped with the Alpine patch that was the most recent one publicly available on its build date (e.g. v2.5.3 was built 2026-04-20, five days after Alpine 3.23.4 published on 2026-04-15; v2.6.5 and v2.7.2 were both built 2026-10-26, three days after Alpine 3.24.3 published on 2026-10-23). None of the "stale-looking" gaps in Table 3 reflect a missed update: v2.6.4 and v2.7.1 (built 2026-10-01) stayed on 3.24.2 because 3.24.3 did not exist until three weeks later. This indicates disciplined, continuous base-image rebasing by the OpenBao maintainers rather than periodic/lagging updates.

6 Security Evolution Analysis (v2.4.0 to v2.7.2)

Throughout this section versions are ordered by version number, not by date. Since v2.6.3 and v2.7.0 shipped together, the 2.6 and 2.7 lines have been patched in parallel, so v2.6.5 (2026-10-26) is *newer* than v2.7.0 (2026-09-23). The upward step from 2.6.5 to 2.7.0 in the charts below is a change of line, not a regression.

6.1 Vulnerability Evolution Table

Table 4: Vulnerability Counts: v2.4.0 through v2.7.2

Version	Critical	High	Medium	Low	Unknown
2.4.0	9	88	98	67	6
2.4.1	9	88	98	67	6
2.4.3	9	87	81	64	6
2.4.4	9	87	79	64	6
2.5.0	9	80	68	45	6
2.5.1	8	79	68	45	6
2.5.2	6	78	67	43	6
2.5.3	1	54	56	38	6
2.5.4	1	48	53	38	6
2.5.5	1	32	35	16	5
2.6.0	0	31	32	14	5
2.6.1	0	29	32	14	5
2.6.2	0	19	31	14	5
2.6.3	0	12	20	2	5
2.6.4	0	12	20	2	5
2.6.5	0	7	6	0	3
2.7.0	0	7	18	1	5
2.7.1	0	7	18	1	5
2.7.2	0	2	4	0	3

6.2 Overall Security Trend Chart

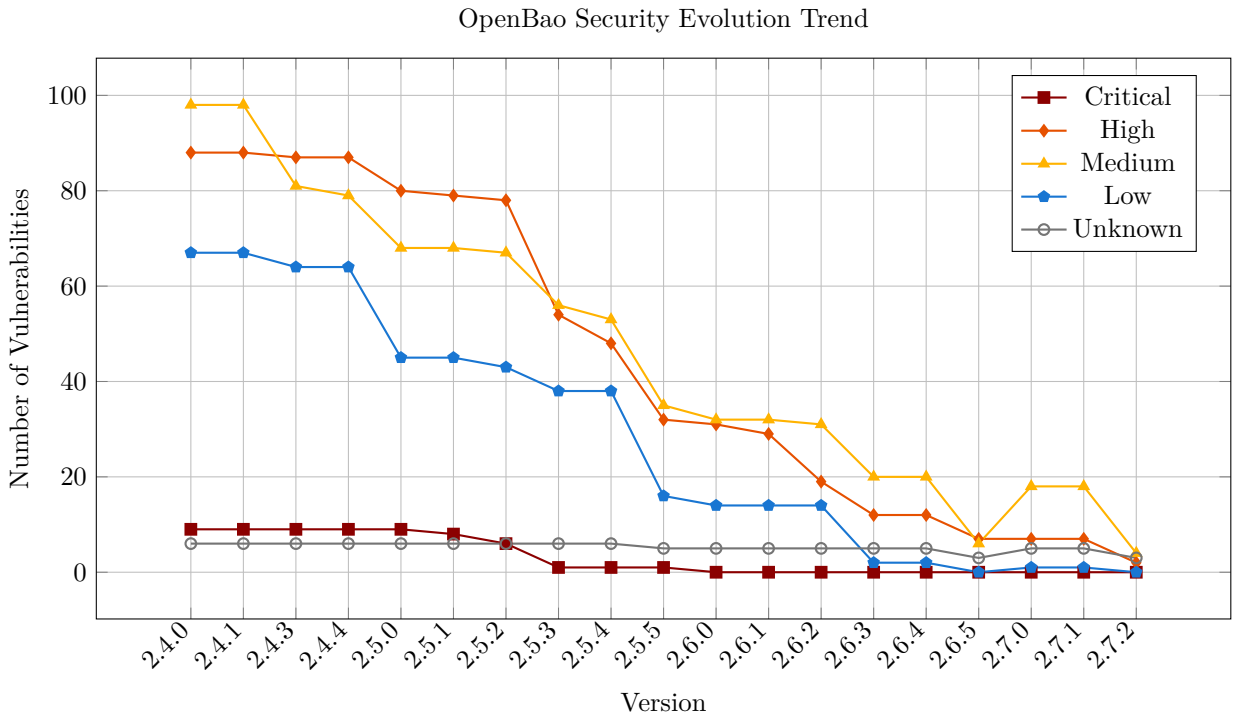


Figure 1: Security Improvement Trend: v2.4.0 to v2.7.2 (version order; 2.6.x and 2.7.x are parallel lines)

6.3 Stacked Area Chart: Total Vulnerability Composition

This stacked area chart provides a complementary view of the security evolution, showing the total volume and composition of vulnerabilities by severity level.

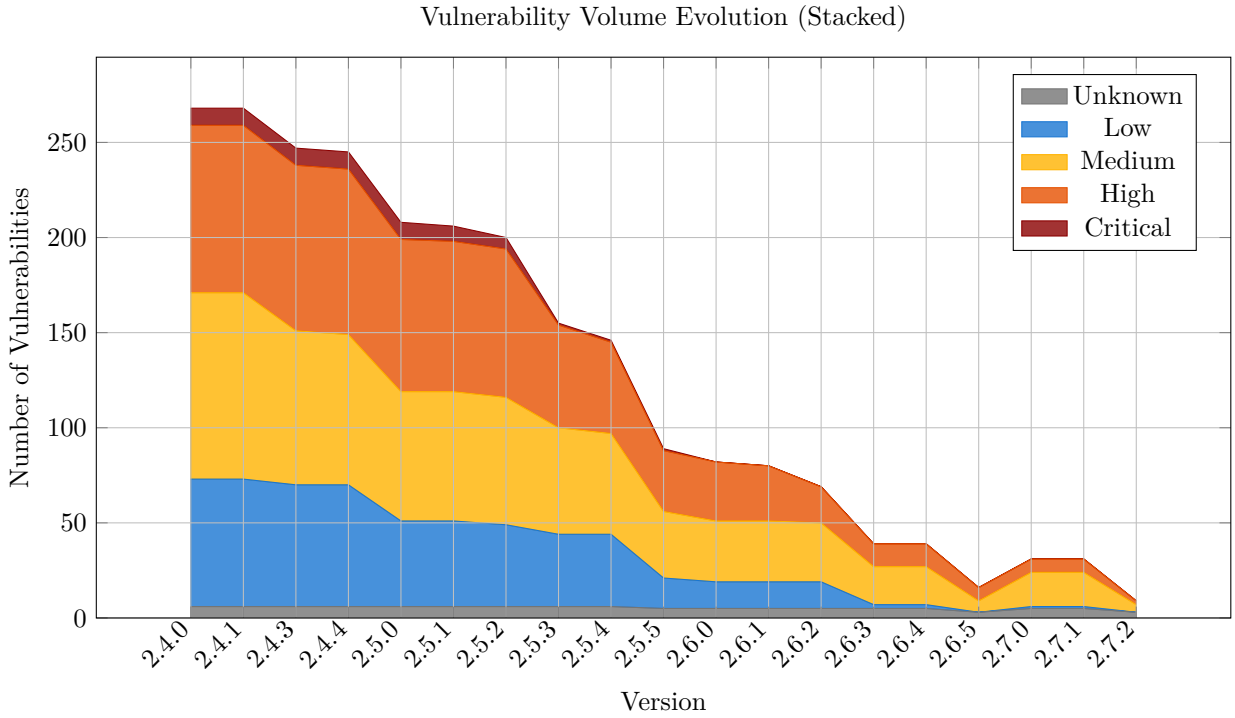


Figure 2: Stacked view showing total vulnerability volume and severity composition

6.4 Version-to-Version Improvement Rate

The following table quantifies the percentage reduction in total vulnerabilities between consecutive releases *of the same line*, highlighting the most impactful updates. The 2.7 line is measured from its branch point, v2.6.3.

Table 5: Vulnerability Reduction Rate Between Versions

Version Transition	Before	After	Reduction
2.4.0 → 2.4.1	268	268	0.0%
2.4.1 → 2.4.3	268	247	7.8%
2.4.3 → 2.4.4	247	245	0.8%
2.4.4 → 2.5.0	245	208	15.1%
2.5.0 → 2.5.1	208	206	1.0%
2.5.1 → 2.5.2	206	200	2.9%
2.5.2 → 2.5.3	200	155	**22.5%**
2.5.3 → 2.5.4	155	146	5.8%
2.5.4 → 2.5.5	146	89	**39.0%**
2.5.5 → 2.6.0	89	82	7.9%
2.6.0 → 2.6.1	82	80	2.4%
2.6.1 → 2.6.2	80	69	13.8%
2.6.2 → 2.6.3	69	39	**43.5%**
2.6.3 → 2.6.4	39	39	0.0%
2.6.4 → 2.6.5	39	16	**59.0%**
2.6.3 → 2.7.0	39	31	20.5%
2.7.0 → 2.7.1	31	31	0.0%
2.7.1 → 2.7.2	31	9	**71.0%**

The data reveals six major security milestones:

- **v2.5.3:** 22.5% reduction - Alpine 3.23.4 rebase plus a large application-layer cut (Critical drops from 6 to 1)
- **v2.5.5:** 39.0% reduction - Alpine upgrade to 3.24.1 more than halves the OS layer (53 → 23)
- **v2.6.3:** 43.5% reduction - The Alpine 3.24.2 rebase removes 20 of the 23 remaining OS-layer findings, clearing the OpenSSL `libcrypto3`/`libssl3` cluster in a single step
- **v2.7.0:** 20.5% reduction against its branch point v2.6.3 — entirely explained by the seven pseudo-version false positives disappearing (Section 7.2), plus one further application finding
- **v2.6.5:** 59.0% reduction - Go stdlib, `x/net` and Alpine 3.24.3 refresh on the 2.6 line
- **v2.7.2:** 71.0% reduction - The same refresh on the 2.7 line, clearing every stdlib, HTTP/2 and OS finding and leaving 9

The two zero-change patches, v2.6.4 and v2.7.1 (both 2026-10-01), shipped on an unchanged Alpine base and Go toolchain; whatever they fixed is not visible to a dependency scanner.

7 Deep Security Insights

This section provides a more granular analysis of the vulnerability data, distinguishing between different layers of the container image and identifying persistent risks.

7.1 OS vs. Application Vulnerabilities

Distinguishing between vulnerabilities in the base operating system (Alpine Linux) and the application layer (Go binaries and dependencies) is crucial for identifying the primary source of risk.

Table 6: Vulnerability Distribution: OS Layer vs. Application Layer

Version	OS (Alpine)	Application (Go)	Total
2.4.0	114	154	268
2.4.1	114	154	268
2.4.3	108	139	247
2.4.4	108	137	245
2.5.0	78	130	208
2.5.1	78	128	206
2.5.2	78	122	200
2.5.3	53	102	155
2.5.4	53	93	146
2.5.5	23	66	89
2.6.0	23	59	82
2.6.1	23	57	80
2.6.2	23	46	69
2.6.3	3	36	39
2.6.4	3	36	39
2.6.5	0	16	16
2.7.0	3	28	31
2.7.1	3	28	31
2.7.2	0	9	9

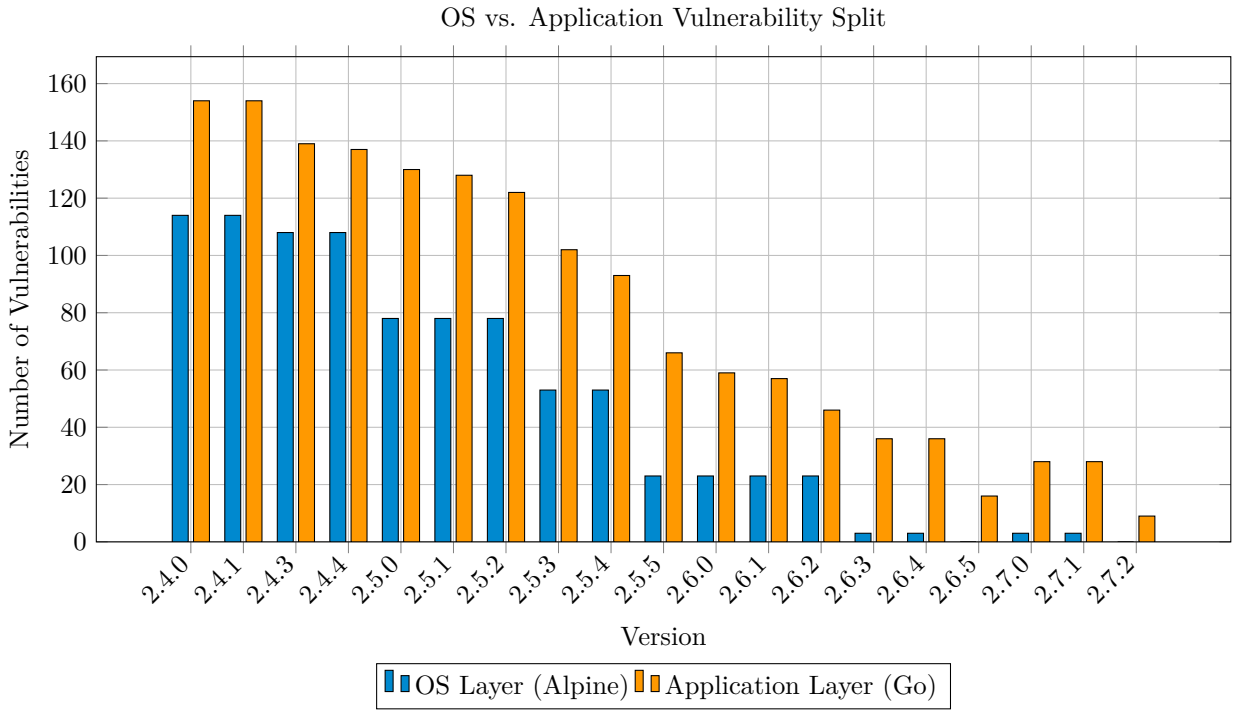


Figure 3: Visualizing the migration of risk from OS to Application layer

The OS layer declines steadily across the series: 114 (v2.4.0) → 78 (v2.5.0–v2.5.2) → 53 (v2.5.3/v2.5.4) → 23 (v2.5.5–v2.6.2) → 3 (Alpine 3.24.2: v2.6.3, v2.6.4, v2.7.0, v2.7.1) → **0** (Alpine 3.24.3: v2.6.5, v2.7.2). The three findings that 3.24.2 still carries against today's database — a `zlib` heap overflow (CVE-2026-85091) and a `musl` advisory (CVE-2026-107891) reported for two packages — were all disclosed after the 3.24.2

builds and fixed by 3.24.3. The application layer follows its own downward path, from 154 findings in v2.4.0 to 9 in both v2.6.5 (after removing false positives) and v2.7.2. With the OS layer clear, the residual surface is a handful of third-party Go modules.

7.2 Scanner Artifact: Pseudo-Version False Positives

Earlier editions of this report credited v2.7.0 with "retiring" seven long-standing OpenBao core advisories (CVE-2024-8185, CVE-2024-9180, CVE-2025-59043, CVE-2025-64761, CVE-2026-45808, CVE-2026-46358, CVE-2026-46405). Inspecting the module metadata Trivy extracts from `/usr/bin/bao` shows that this was not a code fix:

Image	Root Go module	Version recorded in binary
v2.6.3	`github.com/openbao/openbao`	`v0.0.0-20260923163753-63a65e6b9075`
v2.6.5	`github.com/openbao/openbao`	`v0.0.0-20261026193202-b814361ea234`
v2.7.2	`github.com/openbao/openbao/v2`	`v2.7.2`

Every 2.x build through v2.6.5 records its own root module under a Go *pseudo-version* (`v0.0.0-<timestamp>-<commit>`). The advisories above state fixed versions in semver form (2.0.3, 2.4.1, 2.4.4, 2.5.4), and `v0.0.0-…` sorts below every one of them, so Trivy reports all seven against every release regardless of whether the release contains the fix. From v2.7.0, the module path became `github.com/openbao/openbao/v2` with a real semver version; the advisories are keyed on the old path and simply no longer match. Where an advisory's fixed version is itself a pseudo-version (as for most older core advisories), the comparison is like-for-like and remains valid.

Taking each advisory's *own* fixed version at face value, the false positives per release are:

Table 7: Core advisories already fixed in the release that Trivy flags, and corrected figures

Version	Raw Total	False Positives	Corrected Total	Raw Score	Corrected Score
2.4.0	268	2 (2 H)	266	20.7	21.7
2.4.1	268	3 (3 H)	265	20.7	22.2
2.4.3	247	3 (3 H)	244	24.9	26.4
2.4.4	245	4 (4 H)	241	25.3	27.3
2.5.0–2.5.2	208–200	4 (4 H)	204–196	32.9–37.3	34.9–39.3
2.5.3	155	4 (4 H)	151	57.0	59.0
2.5.4	146	7 (5 H, 2 M)	139	60.6	63.5
2.5.5	89	7 (5 H, 2 M)	82	74.4	**77.3**
2.6.0	82	7 (5 H, 2 M)	75	76.7	79.6
2.6.1	80	7 (5 H, 2 M)	73	77.7	80.6
2.6.2	69	7 (5 H, 2 M)	62	82.9	85.8
2.6.3 / 2.6.4	39	7 (5 H, 2 M)	32	89.8	92.7
2.6.5	16	7 (5 H, 2 M)	**9**	95.3	**98.2**
2.7.x	—	0	31 / 31 / 9	—	92.8 / 92.8 / 98.2

Three consequences follow. First, **the 2.6 and 2.7 lines are equally secure**: v2.6.3 (92.7 corrected) matches v2.7.0 (92.8), and v2.6.5 and v2.7.2 carry the *identical* nine findings and score. Second, the correction moves v2.5.5 from the MEDIUM to the LOW risk band. Third, the methodological lesson: a change in a scanner's count can come from a change in *how the artifact identifies itself* rather than in what it contains, and only module-level inspection (`trivy --list-all-pkgs`) distinguishes the two. Main tables in this report keep raw Trivy figures for consistency with earlier editions.

7.3 Remediation Status (Fixable Vulnerabilities)

A "Fixable" vulnerability is one for which a patched version of the affected package is already available. Analyzing this metric helps measure the potential for immediate risk reduction.

Table 8: Fixable vs. Total Vulnerabilities

Version	Fixable	Total	% Fixable
2.4.0	261	268	97%
2.4.1	261	268	97%
2.4.3	240	247	97%
2.4.4	238	245	97%
2.5.0	203	208	98%
2.5.1	201	206	98%
2.5.2	195	200	98%
2.5.3	150	155	97%
2.5.4	141	146	97%
2.5.5	84	89	94%
2.6.0	78	82	95%
2.6.1	76	80	95%
2.6.2	65	69	94%
2.6.3	35	39	90%
2.6.4	35	39	90%
2.6.5	12	16	75%
2.7.0	27	31	87%
2.7.1	27	31	87%
2.7.2	5	9	56%

Across v2.4.0–v2.6.2, 94–98% of identified vulnerabilities have an available fix, which is precisely why upgrading is so effective: almost every finding in an older image is one the maintainers have already addressed downstream. (For v2.5.4–v2.6.5 the seven pseudo-version false positives are counted as "fixable", since Trivy reports a fixed version for them.) The ratio falls to 56% in v2.7.2 because the fixable backlog is nearly gone: of its nine findings, five have upstream fixes (`go-jose/v4` 4.1.5/4.1.6 and `golang.org/x/net` 0.62.0) and four do not (`go-jose/v3` has no patched release, and GO-2026-5932 has none by design).

7.4 Top Recurring CVEs

Identifying vulnerabilities that persist across multiple releases helps highlight long-standing security challenges or dependencies.

Table 9: Most persistent CVEs/advisories across the nineteen analyzed versions

CVE ID	Package	Persistence (Releases)
GO-2026-5932	golang.org/x/crypto	**19 (all)**
CVE-2026-116585	go-jose/v3, v4	**19 (all)**
CVE-2026-116586	go-jose/v3, v4	**19 (all)**
CVE-2026-116587	go-jose/v3, v4	**19 (all)**
CVE-2026-78661	golang.org/x/net	**19 (all)**
CVE-2026-97045	golang.org/x/net	**19 (all)**
CVE-2024-8185 and 6 other core advisories	openbao (core)	16 (2.4.0 to 2.6.5)[†]

[†] False positives from v2.5.4 onward and partially before; see Section 7.2.

Six items now reach maximum persistence, and they are exactly the nine findings of v2.7.2 (the `go-jose` CVEs are each reported once for the v3 and once for the v4 module). Persistence here does *not* mean neglect: the three `go-jose` CVEs were published on **2026-10-26**, the day v2.6.5 and v2.7.2 were built, so no analyzed release could have fixed them. They appear in all nineteen images only because every image ships the affected `go-jose` versions. The two `x/net` HTML-parser advisories require `x/net` 0.62.0, one minor above the 0.61.0

in v2.7.2. GO-2026-5932, as before, flags `golang.org/x/crypto/openpgp` as unmaintained and unsafe by design; no fixed version exists and no upgrade can clear it.

7.5 Detailed Analysis of the Remaining Findings in v2.7.2

Understanding the nature and impact of what remains in the latest release indicates where the next improvements will come from.

Table 10: The nine findings of v2.7.2 (identical to corrected v2.6.5)

CVE ID	CVSS	Severity	Fixed in	Description
CVE-2026-116587	7.5	HIGH	v4: 4.1.6; v3: none	`go-jose`: Denial of Service via crafted RSA parameters in JWKs
CVE-2026-116585	N/A	MEDIUM	v4: 4.1.5; v3: none	`go-jose`: Denial of Service via empty ciphertext buffer during JWE decryption
CVE-2026-116586	N/A	MEDIUM	v4: 4.1.5; v3: none	`go-jose`: Denial of Service via deeply nested JSON parsing
CVE-2026-78661	N/A	UNKNOWN	0.62.0	`x/net/html`: quadratic string reallocation when coalescing text nodes
CVE-2026-97045	N/A	UNKNOWN	0.62.0	`x/net/html`: incorrect integration-point handling in `childTextNodesAreLiteral`
GO-2026-5932	N/A	UNKNOWN	—	`openpgp` package unmaintained and unsafe by design — **no fix available**

All remaining findings are denial-of-service or parser-robustness issues in third-party Go modules; none is in OpenBao's own code. The two upgrades that would clear five of the nine — `go-jose/v4` $\geq$ 4.1.6 and `x/net` $\geq$ 0.62.0 — are routine dependency bumps that can be expected in the next patch. The `go-jose/v3` findings require dropping the v3 module (no fixed v3 release exists), and GO-2026-5932 requires migrating off `openpgp`; both are design work, not patching. Organizations that do not exercise OpenPGP code paths can reasonably treat GO-2026-5932 as informational.

7.6 Release Timeline and Update Velocity

Understanding the release cadence helps assess the project's responsiveness to security issues.

Table 11: OpenBao Release Timeline (2025-2026)

Version	Release Date	Days Since Previous	Total Vulnerabilities
2.4.0	2025-08-28	—	268
2.4.1	2025-09-11	14	268
2.4.3	2025-10-22	41	247
2.4.4	2025-11-24	33	245
2.5.0	2026-02-04	72	208
2.5.1	2026-02-23	19	206
2.5.2	2026-03-25	30	200
2.5.3	2026-04-20	26	155
2.5.4	2026-05-20	30	146
2.5.5	2026-06-17	28	89
2.6.0	2026-07-14	27	82
2.6.1	2026-07-22	8	80
2.6.2	2026-08-18	27	69
2.6.3	2026-09-23	36	39
2.7.0	2026-09-23	0	31
2.6.4	2026-10-01	8	39
2.7.1	2026-10-01	0	31
2.6.5	2026-10-26	25	16
2.7.2	2026-10-26	0	9

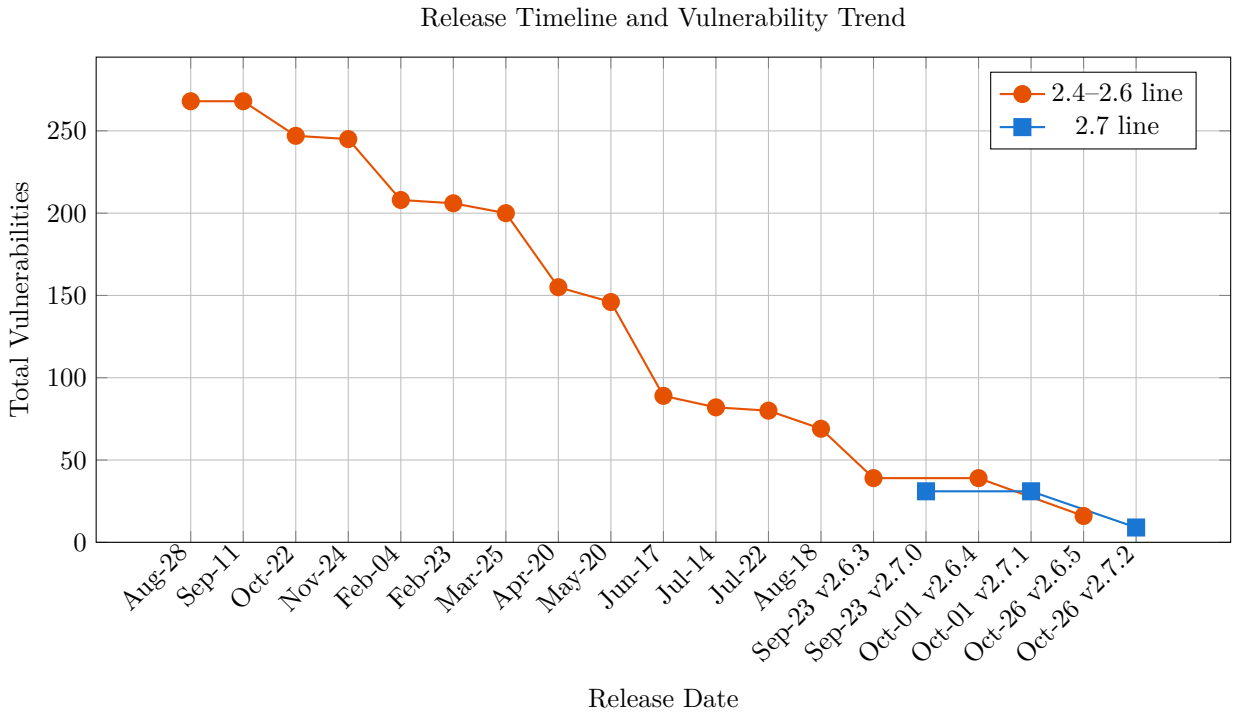


Figure 4: Release frequency and vulnerability reduction over time

The timeline reveals a consistent release cadence averaging ∼28 days across the 2.5.x and 2.6.x series, punctuated by faster turnarounds when warranted. Since 2026-09-23, OpenBao has released **in pairs**: v2.6.3/v2.7.0, v2.6.4/v2.7.1 (eight days later) and v2.6.5/v2.7.2 (25 days after that), each pair built within minutes of each other on the same Alpine base and Go toolchain. Users on either line receive the same OS-

and dependency-level fixes on the same day — the 2.6 line even beyond the 2026-10-01 EOL date that geol records for it.

8 Measuring CVE Database Drift

Every edition of this report has warned that vulnerability counts move over time as new CVEs are disclosed. Because all scan results are committed to version control, **git history holds the same container images measured at several points in time**. Comparing an image against itself across scan dates isolates database drift exactly, since the image is byte-identical and any movement must come from the database alone.

The scan date below is read from the `CreatedAt` field inside each committed scan file rather than from the commit date, so results are attributed to the day they were produced. The analysis is reproducible with `./drift_analysis.sh`.

Table 12: The same image, measured repeatedly: apparent vulnerability count by scan date

Image	Scan Date	Trivy	Total	Crit	High	Med	Low
v2.4.0	2026-05-21	0.70.0	125	9	45	58	13
v2.4.0	2026-06-22	0.71.2	165	7	49	57	52
v2.4.0	2026-07-16	0.72.0	195	8	63	68	52
v2.4.0	2026-07-23	0.72.0	196	8	64	68	52
v2.4.0	2026-09-24	0.74.0	235	9	79	79	66
v2.4.0	2026-11-01	0.76.1	**268**	9	88	98	67
v2.5.5	2026-06-18	0.71.1	9	0	5	3	1
v2.5.5	2026-06-22	0.71.2	9	0	5	3	1
v2.5.5	2026-07-16	0.72.0	14	0	6	4	1
v2.5.5	2026-07-23	0.72.0	15	0	7	4	1
v2.5.5	2026-09-24	0.74.0	55	1	23	15	15
v2.5.5	2026-11-01	0.76.1	**89**	1	32	35	16
v2.6.1	2026-07-23	0.72.0	10	0	6	3	0
v2.6.1	2026-09-24	0.74.0	47	0	20	13	13
v2.6.1	2026-11-01	0.76.1	**80**	0	29	32	14
v2.7.0	2026-09-24	0.74.0	1	0	0	0	0
v2.7.0	2026-11-01	0.76.1	**31**	0	7	18	1

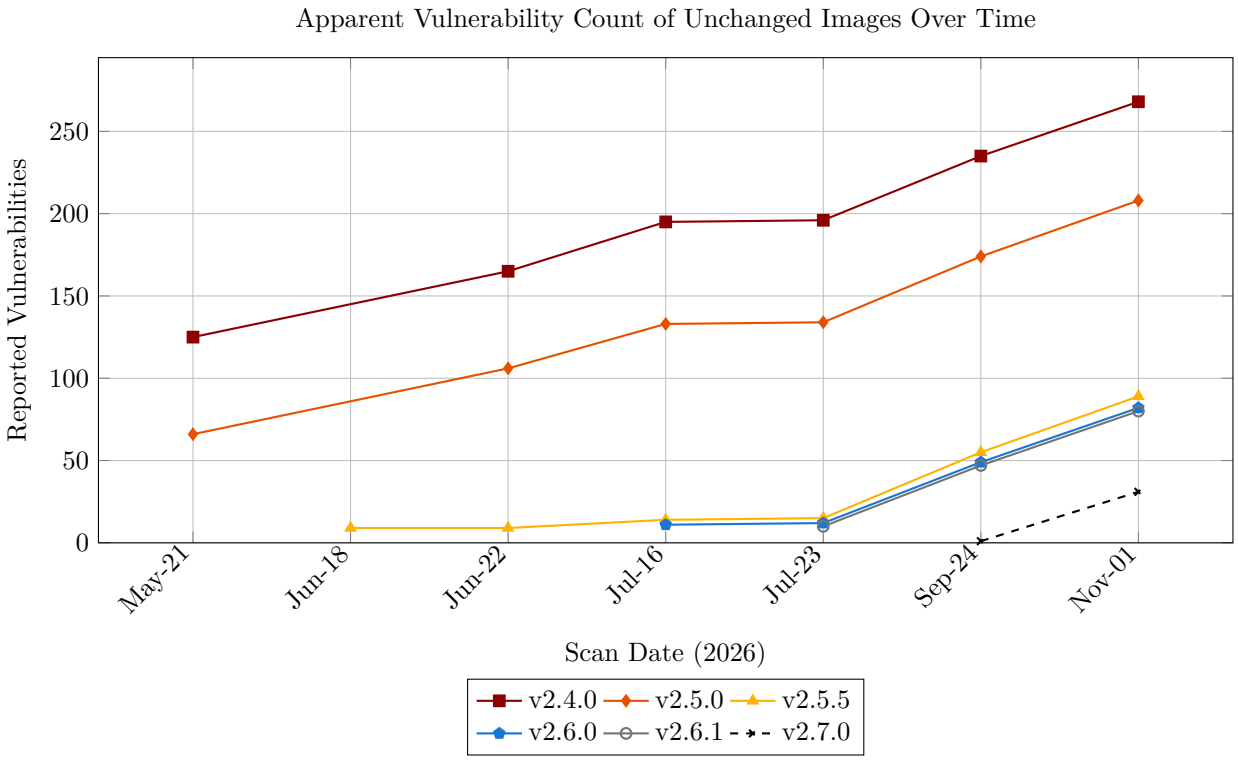


Figure 5: Every line is a *fixed, unchanged* image. All upward movement is database drift.

Not one of these images was rebuilt, yet every line rises. Table 13 quantifies the effect.

Table 13: Drift magnitude per image over its observed window

Image	Window (days)	First	Latest	Change	Per 30 days
v2.4.0	164	125	268	+143 (+114%)	+26.2
v2.5.0	164	66	208	+142 (+215%)	+26.0
v2.5.5	136	9	89	+80 (+889%)	+17.6
v2.6.0	108	11	82	+71 (+645%)	+19.7
v2.6.1	101	10	80	+70 (+700%)	+20.8
v2.7.0	38	1	31	+30 (+3000%)	+23.7

An unchanged OpenBao image accumulates roughly **18–26 newly-disclosed findings per month**. Older images drift faster in absolute terms — they carry more packages with more surface — while newer, leaner images drift far more dramatically in *relative* terms, because they start from a small base. The extreme case is v2.7.0: published in the previous edition with a single finding, it reports 31 five weeks later, a 31-fold increase with no change whatsoever to the artifact. v2.5.5 was published at 9 findings and stands at 89 today.

8.1 Decomposing an Apparent Regression

The previous edition hypothesized that drift "could equally have disguised a genuine regression as progress". This edition observes the mirror image: **drift disguising genuine progress as a regression**. The previous edition's best version, v2.7.0, was published at 1 finding; this edition's best, v2.7.2, stands at 9. A reader comparing the two editions would conclude that the 2.7 line got worse.

Component	Findings	What it measures
Naive cross-edition delta	+8	v2.7.0 as published vs. v2.7.2 now
Real improvement	**−22**	v2.7.0 vs. v2.7.2, *both* scanned 2026-11-01
Database drift	+30	v2.7.0 vs. itself, 2026-09-24 → 2026-11-01
Sum	+8	reconciles to the naive figure

The engineering work in v2.7.1/v2.7.2 removed **22** findings, yet the published figure rose by 8, because 30 newly-disclosed CVEs landed against the baseline in the same five weeks. The sign of the naive comparison is simply wrong. Combined with the previous edition's case (a real improvement of 46 that looked like 9), this establishes that cross-edition count comparisons cannot be trusted for either magnitude or direction.

8.2 Consequences for Reading This Report

Three practical rules follow:

1. **Comparisons within this report are valid.** All nineteen tags were scanned in a single session against one database, so version-to-version differences reflect the images alone (subject to the pseudo-version correction of Section 7.2).
2. **Comparisons against externally published counts are not valid**, including against earlier editions of this report, unless the scan date and database version match.
3. **An absolute vulnerability count is meaningless without its measurement date.** "v2.7.0 has 1 vulnerability" was true on 2026-09-24 and is wrong today by a factor of thirty, with no dishonesty and no change to the image.

This is also the operational argument for upgrading on a schedule rather than on evidence: a deployed image does not become more secure by standing still, but the measured risk attached to it grows steadily. The drift rates above suggest an unchanged OpenBao deployment accrues on the order of 20–25 newly-disclosed findings per month of inaction.

9 Glossary of Security Terms

CVE (Common Vulnerabilities and Exposures): A list of publicly disclosed computer security flaws. Each entry (e.g., CVE-2024-1234) represents a specific vulnerability in a software package.

CVSS (Common Vulnerability Scoring System): A free and open industry standard for assessing the severity of computer system security vulnerabilities. Scores range from 0 to 10.

Severity Levels:
- **Critical:** Vulnerabilities that could be easily exploited by a remote attacker to gain full control of the system.
- **High:** Vulnerabilities that are difficult to exploit but could result in significant elevated privileges or data loss.
- **Medium:** Vulnerabilities that require specific configurations or local access to exploit.
- **Low:** Vulnerabilities that have very little impact on the system's security.

Go pseudo-version: A version string of the form `v0.0.0-<timestamp>-<commit>` that Go records for a module built without a semver tag. It sorts below any real release, which can cause scanners to match advisories that the code has already fixed.

OCI Image (Open Container Initiative): A standard for container images that ensures portability across different container engines (like Docker or Podman).

Trivy: An open-source vulnerability scanner specifically designed for containers and other artifacts.

10 Conclusion

The analysis indicates that OpenBao has undergone sustained and substantial security hardening. Moving from version 2.4.0 to the latest 2.7.2 release reduces the vulnerability count from 268 to 9 — a **96.6% reduction** — with no Critical or Low findings left, two High and four Medium, and a 98.2/100 weighted Security Posture Score.

Two distinct forces produced that result. The **operating-system layer** was cleared by disciplined base-image maintenance: all nineteen analyzed releases shipped the newest Alpine patch available on their build date, and the Alpine 3.24.3 rebase in v2.6.5/v2.7.2 brings the OS layer to zero against today's database. The **application layer** was cleared by steady Go toolchain and dependency upgrades; what remains is three `go-jose` denial-of-service CVEs disclosed on the day of the latest build, two `x/net` HTML-parser advisories one minor version away from a fix, and the unfixable GO-2026-5932.

This edition also **corrects** the previous one. The seven OpenBao core advisories it described as retired by v2.7.0 were a scanner artifact: up to v2.6.5, the `bao` binary records its own module under a Go pseudo-version that sorts below every fixed version, and v2.7.0 merely renamed the module to `github.com/openbao/openbao/v2`. With that artifact removed, **v2.6.5 and v2.7.2 are identical in vulnerability terms** (nine findings, 98.2/100).

OpenBao's adherence to Semantic Versioning and its low-risk minor release strategy — similar to PostgreSQL's versioning philosophy — ensures that upgrading to the latest minor version is not only safe but actively recommended. The paired releases since 2026-09-23 show the project serving conservative users on the 2.6 line and those moving to 2.7 with the same fixes on the same day.

One methodological point deserves emphasis. Every version's counts are higher than in the previous edition, purely because this rescan used a database five weeks newer: v2.7.0 moved from 1 finding to 31 with no change to the image. Comparing editions naively would show v2.7.2 (9) as a regression from v2.7.0 (1); scanning both on the same day shows a real improvement of 22, masked by 30 findings of drift. Relative comparisons within this report remain valid; comparisons against externally published figures, earlier editions of this report included, do not.

Organizations should run **v2.6.5 or v2.7.2**. Given the 2.6 cycle's listed EOL of 2026-10-01, v2.7.2 is the better long-term target. This rescan places v2.4.0 through v2.5.2 in the HIGH risk band, v2.5.3 through v2.5.5 in MEDIUM, and v2.6.0 onward in LOW. As demonstrated throughout, staying on an outdated minor version exposes systems to substantially higher risk than the minimal effort a minor upgrade requires.